

class ListOfAdopters

A `ListOfAdopters` is a list of `Adopter` objects that stores information for potential adopters at the pet store and can evaluate criteria based on the preferences of the adopters.

All methods in `ListOfAdopters` is Read-Only except for `new_adopter`, which adds another `Adopter` object to the internal `adopters` list. Indexing of `list` objects begins at 0 while ranking begins at 1.

`__init__()`

Initializes with an empty list of adopters called `adopters`

`new_adopter(preferences)`

Creates a new `Adopter` object by calling to store the preferences of a potential adopter and adds the object to the list of potential adopters.

preferences (<code>list[int]</code>)	A list of pet indexes in preference order: n-1 index is nth preference.
Returns	The index of the new adopter in <code>self.adopters</code> .
Raises	<code>TypeError</code> if the inputted <code>preferences</code> object is not a list of integers.

```
new_adopter([1st preference, 2nd preference, ..., nth preference])
```

`pos_favorites()`

Decides weather it is possible to have a matching where every adopter is matched with their favorite pet.

Returns	<code>True</code> if there is a possible matching where every adopter is matched with their 1st choice pet (when every adopter's first choice is distinct from the other adopters' first choices). Otherwise returns <code>False</code> .
Calls	<code>Adopter.get_preferences()</code> to check for overlap among adopters' first preference.

`pet_popularity(adopters)`

Ranks the pets based on their popularity among adopters in the input adopter parameter. The method ranks only based on the `Adopter` objects passed in through the `adopters` list

adopters (<code>list[Adopter]</code>)	A list of adopter objects.
--	----------------------------

Returns	A ranked list of pet ids with the most popular pet first.
Raises	<code>TypeError</code> if <code>adopters</code> is not specifically a list of adopter objects.

`all_favorites(matching)`

Determines if every adopter received their favorite pet.

matching	A <code>Matching</code> object that contains pairs of adopter indexes and pet indexes. Assumes <code>matching</code> contains no duplicate pairs that contain the same <code>Adopter</code> or pet.
Returns	<code>True</code> if every adopter in <code>matching</code> has their favorite pet. Otherwise returns <code>False</code> .
Raises	<code>TypeError</code> if <code>matching</code> is not a <code>Matching</code> object.
Calls	<code>Adopter.get_preferences()</code> to compare each adopter's match with their first preference.

`all_matched(matching)`

Determines if every adopter in `self.adopters` received a pet in `matching`.

matching	A <code>Matching</code> object that contains pairs of adopter indexes and pet indexes. Assumes <code>matching</code> contains no duplicate pairs that contain the same <code>Adopter</code> or pet.
Returns	<code>True</code> if every adopter in <code>self.adopters</code> is in <code>matching</code> with a pet. Otherwise returns <code>False</code> .
Raises	<code>TypeError</code> if <code>matching</code> is not a <code>Matching</code> object.

`top_n(matching, n)`

Determines whether every adopter receives a pet from among their top n choices.

matching	A <code>Matching</code> object that contains pairs of adopter indexes and pet indexes. Assumes <code>matching</code> contains no duplicate pairs that contain the same <code>Adopter</code> or pet.
n (int)	an integer value that determines the how many preferences to consider for a "successful" match. $0 \leq n \leq \text{len}(\text{preferences})$.
Returns	<code>True</code> if each adopter is matched with a pet from their top n choices. Otherwise returns <code>False</code> .
Raises	<code>TypeError</code> if <code>matching</code> is not a <code>Matching</code> object, or <code>IndexError</code> if <code>n</code> is out of range.
Calls	<code>Adopter.top_npreferences()</code> to compare each adopter's match with their top n preferences.